



Universidade Federal de Santa Catarina – UFSC
DEC – Engenharia de Computação

Linguagem de Descrição de Hardware – Unidade 5

Dr. Ney Laert Vilar Calazans

Última alteração: 24/06/2026

❖ Baseado em materiais originais dos Profs. **Ney Calazans**, **Fernando Moraes (PUCRS)**, e **Rafael Garibotti (PUCRS)**

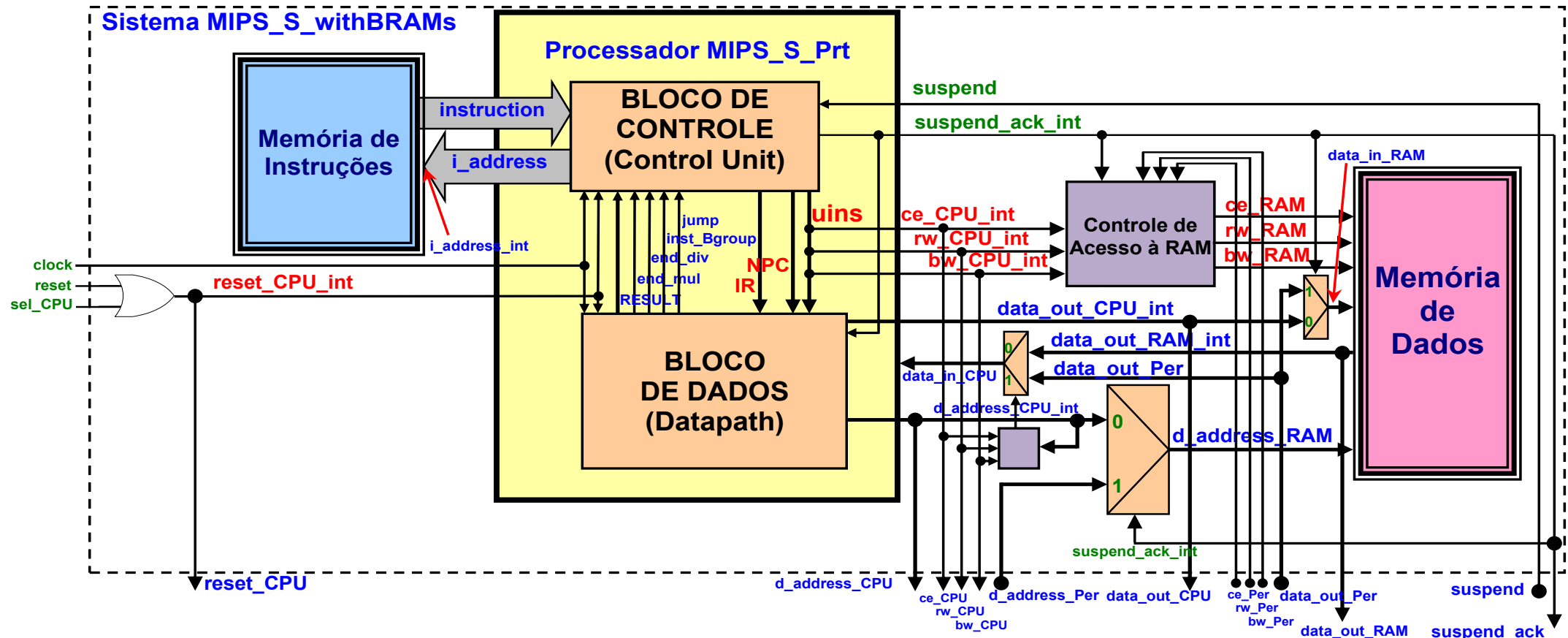
SIMULAÇÃO E PROTOTIPAGEM DE SISTEMAS COMPLEXOS (SISTEMA PROCESSADOR + MEMÓRIAS)

INTRODUÇÃO

- Deve-se simular e prototipar um sistema processador em uma das plataformas de Hw Nexys, empregando o esquema que provê **entrada/saída** de dados pelo sistema processador+memórias
- Disponibiliza-se uma organização funcional simulável e prototipável de um sistema composto por **um processador** e **memórias** de **instruções** e de **dados** e uma **interface de acesso** ao sistema completo (**MIPS_S_withBRAMs**). O sistema baseia-se na arquitetura **MIPS_S**, um subconjunto da arquitetura MIPS32. Esta arquitetura e uma organização para a mesma estão descritas em **material disponibilizado aqui**
- A organização funcional simulável e prototipável mencionada antes é dada. Também será dado um esqueleto de periférico com interface e memória. Deve-se realizar três projetos preparatórios (Sw e Hw) e o Projeto 4, que completa e valida o sistema operando corretamente na plataforma **Nexys 2 (tem que ser esta...)**

INTRODUÇÃO

- Abaixo mostra-se o diagrama de blocos do sistema MIPS_S_Prt+memórias e uma lógica de cola simples → Aqui, este sistema processador é denominado MIPS_S_withBRAMs
- Deve-se simular e prototipar este sistema. Sua interface externa comporta várias classes distintas de periféricos
- **Atenção: Este ambiente foi incrementado em relação a semestres anteriores!!! Vejam a documentação...**



VHDL Parcial do Sistema

MIPS_S_withBRAMs

```
entity MIPS_S_withBRAMs is
    port(clock, reset, selCPU, suspend, ce_Per, rw_Per, bw_Per : in std_logic; d_address_Per, data_out_Per: in wires32; suspend_ack, ce_CPU, rw_CPU, bw_CPU: out std_logic; addressCPU: out wires32;
          d_address_CPU, data_out_CPU, data_out_RAM: out wires32);
end MIPS_S_withBRAMs;
architecture MIPS_S_withBRAMs of MIPS_S_withBRAMs is
    --- vários sinais são declarados aqui (não mostrados, por concisão)
begin
    reset_CPU_int <= reset or sel_CPU;
    reset_CPU <= reset_CPU_int;
    Mem_Clock: IDMem_clock <= not clock;

    P_mem: entity work.program_memory port map
        (clock=>IDMem_clock, address=>i_address_int, instruction=>instruction);

    I_MIPS_S: entity work.MIPS_S port map
        (clock=>clock, reset=>reset_CPU_int, ce=>ce_CPU_int, rw=>rw_CPU_int, bw=>bw_CPU_int, suspend=>suspend, suspend_ack=>suspend_ack_int, ii_address=>i_address, instruction=>instruction,
         d_address=>d_address_CPU_int, data_in=>data_in_CPU, data_out=>data_out_CPU_int);

    I_Ctrl_Sigs: ce_RAM <= ce_CPU_int when (suspend_ack_int='0') else ce_Per; rw_RAM <= rw_CPU_int when (suspend_ack_int='0') else rw_Per;
                bw_RAM <= bw_CPU_int when (suspend_ack_int='0') else bw_Per;
    D_RAM_Addr_Mux: d_address_RAM <= d_address_CPU_int(12 downto 0) when (suspend_ack_int='0') else d_address_Per(12 downto 0);
    D_RAM_Data_Mux: data_in_RAM <= data_out_CPU_int when (suspend_ack_int='0') else data_out_Per;
    CPU_Data_Mux_in: data_in_CPU <= data_out_Per when (ce_CPU_int='1' and rw_CPU_int='1' and (not (d_address_CPU_int >= FIRST_D_ADDRESS and d_address_CPU_int <= LAST_D_ADDRESS)))
                    else data_out_RAM_int;

    D_mem: entity work.data_mem_mod0 port map
        (clock=>IDMem_clock, ce=>ce_RAM, we=>rw_RAM, bw=>bw_RAM, address=>d_address_RAM(12 downto 2), byte_choice=>d_address_RAM(1 downto 0),
         data_in=>data_in_RAM, data_out=>data_out_RAM_int);

    d_address_CPU <= d_address_CPU_int; ce_CPU <= ce_CPU_int; rw_CPU <= rw_CPU_int; bw_CPU <= bw_CPU_int; data_out_CPU <= data_out_CPU_int;
    data_out_RAM <= data_out_RAM_int; suspend_ack <= suspend_ack_int;
end MIPS_S_withBRAMs;
```

Estrutura da Distribuição

- Documentação Geral
- Exemplos de programas em linguagem de montagem (*assembly language*) do MIPS
- Fontes VHDL para simulação (MIPS_S_Sim) e material auxiliar
- Fontes VHDL para prototipação (MIPS_S_Prt) e material auxiliar

```
Distribution_V4.2/
0 Docs
  1-Intro_FPGAS&Simulacao_ISE.pdf
  1-Intro_FPGAS&Simulacao_Tut1_Vivado.pdf
  2-Tutssim-Prot_FPGAs_Tut1Tut2_ISE_Nex12.pdf
  2-Tutssim-Prot_FPGAs_Tut2_ISE-Vivado_NexA7.pdf
  5_0-MIPS_AppA-comErrata_V7.pdf
  5_1-Spec_ArqOrg_MIPS_S_V3.02_Port.pdf
  5_2-MIPS_S-Apresentacao_V11.pdf
  5_3-MIPS_IntroProgramming.pdf
  5_4-MIPS_S_withBRAMS_Presentation_V5.9.pdf
  5_5-Simulacao MIPS_S_withBRAMS_V5.9.pdf
  5_6-ExercPropostos MIPS_S_V1.0.pdf
1 AssemblyExs
  Allinsts_MIPS_S.asm
  mult_div.asm
  SBLB_Tests.asm
  SomaVet.asm
2 MIPS_S_Sim
  README.txt
  Sw_Wcfg_files
    Allinsts_MIPS_S.txt
    mult_div.txt
    Wave-Allinst.wcfg
    Wave-muldiv.wcfg
  VHDL
    MIPS_S_Sim_tb.vhd
    MIPS_S_Sim.vhd
    mult_div.vhd
3 MIPS_S_Prt
  le_MARS
    le_mars.c
    le_mars.exe
    Memory_Srcs
      Allinsts_MIPS_S_data.txt
      Allinsts_MIPS_S_prog.txt
      SBLB_data.txt
      SBLB_prog.txt
      SomaVet_data.txt
      SomaVet_prog.txt
    Memory_VHDLs
      Allinsts_MIPS_S_memory.vhd
      SBLB_memory.vhd
      SomaVet_memory.vhd
  Nexys-A7-100T-Master.xdc
  README.txt
  Sw_Wcfg_files
    LBSB_wform.wcfg
    SomaVet_wform.wcfg
    Wave-Allinst.wcfg
    Wave-muldiv.wcfg
  VHDL
    dspl_drv_Nexys_1_2.vhd
    dspl_drv_Nexys_A7.vhd
    MIPS_S_com_perif_tb.vhd
    MIPS_S_Prt.vhd
    MIPS_S_sem_perif_tb.vhd
    MIPS_S_withBRAMS.vhd
    mult_div.vhd
  README.txt
11 directories, 48 files
```

Um Software Simples - Programa *SomaVet.asm* (em Assembly do MIPS)

```
.data                                # Three 11-element arrays (or vectors) and their size
V2:    .word    0x0      0x0  0x0  0x0  0x0  0x0  0x0  0x0  0x0  0x0  0x0  0x0
V0:    .word    0x1000011 0xff 0x3  0x14 0x878 0x31  0x62 0x10 0x5  0x16 0xAB000002
V1:    .word    0x2000002 0xf4 0x3  0x14 0x878 0x31  0x62 0x10 0x5  0x16 0x21000020
size:   .word    11

.text                                # Add what follows to the text segment of the program
.globl  main                        # Declare the label main to be a global one

main:

    la      $t0,V0                  # generate pointer to V0 source array (pseudo-inst)
    la      $t1,V1                  # generate pointer to V1 source array (pseudo-inst)
    la      $t2,V2                  # generate pointer to V2 destination array (pseudo-inst)
    la      $t3,size                # get address of size (pseudo-inst)
    lw      $t3,0($t3)              # here, register $t3 contains the no. of elements to process

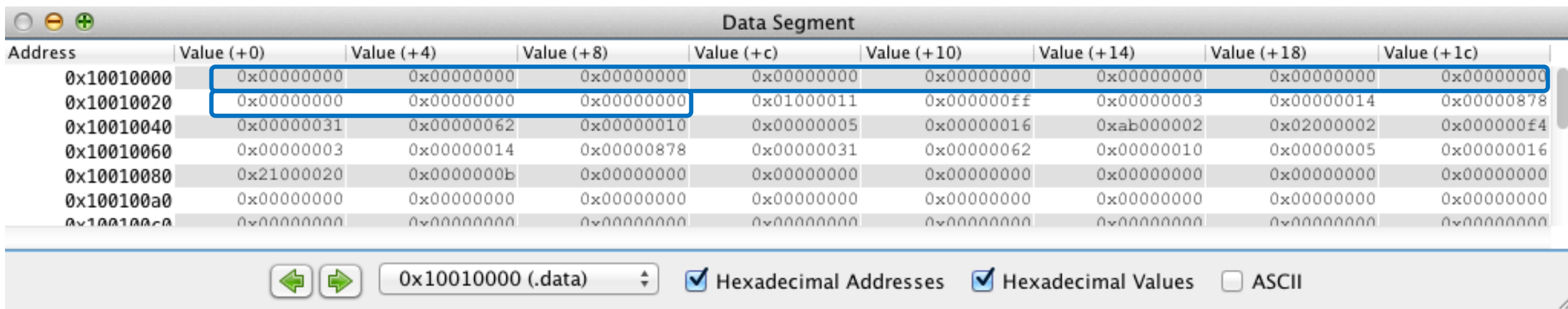
loop:

    blez     $t3,end                # if no. of elements to process is/becomes 0, end of processing
    lw       $t4,0($t0)
    lw       $t5,0($t1)
    addu     $t4,$t4,$t5
    sw       $t4,0($t2)              # update V2 array element in memory
    addiu    $t0,$t0,4               # advance pointers
    addiu    $t1,$t1,4
    addiu    $t2,$t2,4
    addiu    $t3,$t3,-1              # decrement no. of elements to process
    j        loop                   # execute the loop another time

    # now, stop
end:      j        end
```

Validação com o Simulador MARS

- Área de dados antes e depois da soma
 - ✓ V2 começa com todos seus elementos em 0

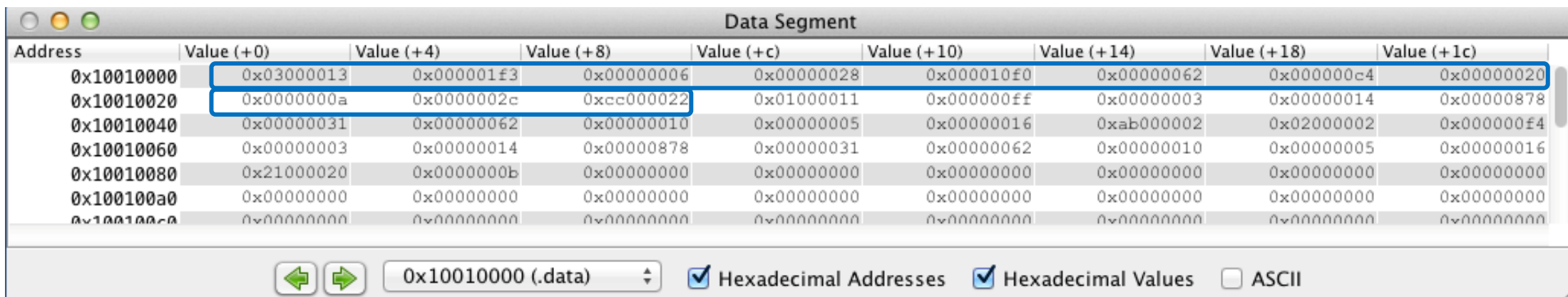


The screenshot shows the MARS Data Segment window with the following data:

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010020	0x00000000	0x00000000	0x00000000	0x01000011	0x000000ff	0x00000003	0x00000014	0x00000878
0x10010040	0x00000031	0x00000062	0x00000010	0x00000005	0x00000016	0xab000002	0x02000002	0x000000f4
0x10010060	0x00000003	0x00000014	0x00000878	0x00000031	0x00000062	0x00000010	0x00000005	0x00000016
0x10010080	0x21000020	0x0000000b	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100a0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100c0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

Below the table, the address field is set to 0x10010000 (.data). The checkboxes for Hexadecimal Addresses and Hexadecimal Values are checked, and the ASCII checkbox is unchecked.

- ✓ Resultado da soma dos vetores V0 e V1 em V2



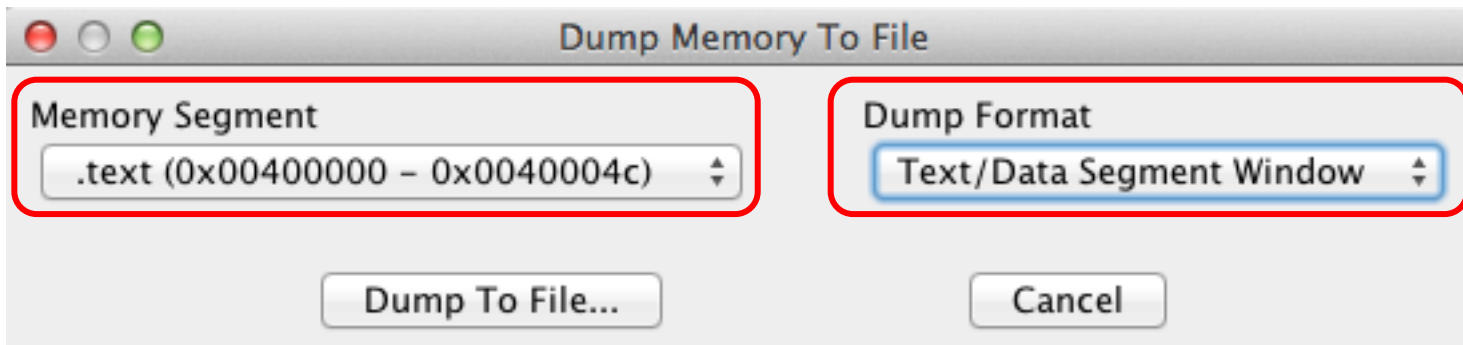
The screenshot shows the MARS Data Segment window with the following data after the addition operation:

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	0x03000013	0x000001f3	0x00000006	0x00000028	0x000010f0	0x00000062	0x000000c4	0x00000020
0x10010020	0x0000000a	0x0000002c	0xcc000022	0x01000011	0x000000ff	0x00000003	0x00000014	0x00000878
0x10010040	0x00000031	0x00000062	0x00000010	0x00000005	0x00000016	0xab000002	0x02000002	0x000000f4
0x10010060	0x00000003	0x00000014	0x00000878	0x00000031	0x00000062	0x00000010	0x00000005	0x00000016
0x10010080	0x21000020	0x0000000b	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100a0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100c0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

Below the table, the address field is set to 0x10010000 (.data). The checkboxes for Hexadecimal Addresses and Hexadecimal Values are checked, and the ASCII checkbox is unchecked.

Gerando os *Dumps* de Memória (Código Objeto)

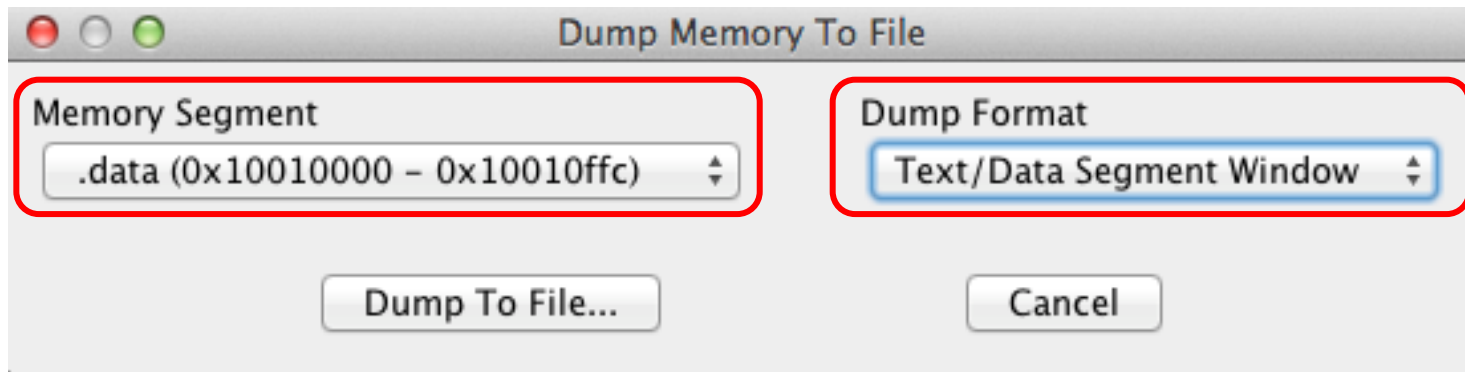
➤ No MARS: File → Dump Memory → arquivo **prog.txt**



Address	Code	Basic	Source
0x00400000	0x3c011001	lui \$1,0x00001001	10 1a \$t0,v1
0x00400004	0x3428002c	ori \$8,\$1,0x0000002c	
0x00400008	0x3c011001	lui \$1,0x00001001	11 1a \$t1,v2
0x0040000c	0x34290058	ori \$9,\$1,0x00000058	
0x00400010	0x3c011001	lui \$1,0x00001001	12 1a \$t2,v3
0x00400014	0x342a0000	ori \$10,\$1,0x00000000	
...			
0x0040004c	0x08100013	j 0x0040004c	30 end: j end

Gerando os *Dumps* de Memória (Dados do Programa)

➤ No MARS: File → Dump Memory → arquivo **data.txt**



```
0x10010000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x10010020 0x00000000 0x00000000 0x00000000 0x01000011 0x000000ff 0x00000003 0x00000014 0x00000878
0x10010040 0x00000031 0x00000062 0x00000010 0x00000005 0x00000016 0xab000002 0x02000002 0x000000f4
0x10010060 0x00000003 0x00000014 0x00000878 0x00000031 0x00000062 0x00000010 0x00000005 0x00000016
..... e continua .....
```

Memórias no FPGA – Instruções

```
library UNISIM;  
use UNISIM.vcomponents.all;
```

```
entity program_memory is  
    port( clock: in std_logic;  
          address: in std_logic_vector(8 downto 0);  
          instruction: out std_logic_vector(31 downto 0));  
end program_memory;
```

```
architecture program_memory of program_memory is  
begin
```

```
    inst_mem : RAMB16_S36  
    generic map (  
        INIT_00 => X"342b00843c011001342a00003c011001342900583c0110013428002d3c011001",  
        INIT_01 => X"2529000425080004ad4c0000018d60218d2d00008d0c0000196000098d6b0000",  
        INIT_02 => X"0000000000000000000000000000000000000000000000000000000000000000",  
        INIT_03 => X"0000000000000000000000000000000000000000000000000000000000000000",  
        INIT_04 => X"0000000000000000000000000000000000000000000000000000000000000000",  
        ...  
        INIT_3D => X"0000000000000000000000000000000000000000000000000000000000000000",  
        INIT_3E => X"0000000000000000000000000000000000000000000000000000000000000000",  
        INIT_3F => X"0000000000000000000000000000000000000000000000000000000000000000",  
    )  
    port map (  
        CLK      => clock,  
        ADDR     => address,  
        EN       => '1',  
        WE       => '0',  
        DI       => x"00000000",  
        DIP      => x"0",  
        DO       => instruction,  
        SSR      => '0'  
    );
```

```
end program_memory;
```

➤ Blocos de 16Kbits. Para as instruções, tem-se 512x32 (2Kbytes)

➤ **Limitação:** programa com no máximo 512 instruções

✓ Interface com o processador

- endereços: 9 bits (512 palavras de 32 bits)
- Instruções de 32 bits

✓ Primeira Instrução do Programa

✓ Inicialização dos 16Kbits através de 64 (0x00 a 0x3F) vetores com strings de 256 bits (64 dígitos hexa por linha)

✓ Sempre habilitada para leitura (EN=1)

✓ A Memória de Instruções nunca é escrita (WE=0)

✓ Ou seja, trata-se de uma ROM

Memórias no FPGA – Dados

- Cada módulo de memória → 4 blocos de 16Kbits (4x2Kbytes = 8Kbytes)
- Memória entrelaçada - Cada bloco contém 8 bits de uma palavra de 32 bits
- Exemplo de conteúdo a armazenar na memória de dados

Endereço	Conteúdo
0x1001000	00 00 00 00
0x1001004	10 00 00 AA
0x1001008	20 00 BB 00
0x100100C	30 CC 00 00

- A inicialização da memória com este conteúdo é (lembrem-se: o simulador e o Hw trabalham com endereçamento “little-endian”. Logo, **mem 0** tem os bytes menos significativos de cada conjunto de 32 bits)

```
mem 0:  INIT_00 => X"00....0000AA00",  
...  
mem 1:  INIT_00 => X"00....00BB0000",  
...  
mem 2:  INIT_00 => X"00....CC000000",  
...  
mem 3:  INIT_00 => X"00....30201000",  
...
```

Memórias no FPGA – Dados

```
library IEEE;
use IEEE.Std_Logic_1164.all;
library UNISIM;
use UNISIM.vcomponents.all;

entity data_mem_mod0 is
    port( clock, ce, -- For now, ce is unused in module,
          -- The EN input of all blocks is '1'.
          we, bw: in std_logic;
          address: in std_logic_vector(10 downto 0); -- 11 bits - 2048 addressable words
          byte_choice: in std_logic_vector(1 downto 0); -- 2 bits - 1 of 4 addressable bytes
          data_in: in std_logic_vector(31 downto 0);
          data_out: out std_logic_vector(31 downto 0));
end data_mem_mod0;
```

Interface com o processador

- Endereço: 11 bits ($2^{11}=2048$)
- **byte_choice** seleciona 1 bloco de 4, se necessário
- Dados: **data in** e **data out**, 32 bits

```
architecture data_mem_mod0 of data_mem_mod0 is
    signal we3, we2, we1, we0 : std_logic;
    signal d_in_mem3, d_in_mem2, d_in_mem1, d_in_mem0 : std_logic_vector(7 downto 0);
begin
```

```
we3 <= '1' when (we='0' and ((bw='0') or (bw='1' and byte_choice="11"))) else '0';
we2 <= '1' when (we='0' and ((bw='0') or (bw='1' and byte_choice="10"))) else '0';
we1 <= '1' when (we='0' and ((bw='0') or (bw='1' and byte_choice="01"))) else '0';
we0 <= '1' when (we='0' and ((bw='0') or (bw='1' and byte_choice="00"))) else '0';

d_in_mem3 <= data_in(7 downto 0) when bw='1' else data_in(31 downto 24);
d_in_mem2 <= data_in(7 downto 0) when bw='1' else data_in(23 downto 16);
d_in_mem1 <= data_in(7 downto 0) when bw='1' else data_in(15 downto 8);
d_in_mem0 <= data_in(7 downto 0);
```

bw controla escrita de word ou byte
(‘1’ ao executar a instrução SB)

Ao escrever apenas 1 byte (instrução SB do MIPS), sempre pega o byte inferior dos 32 bits de entrada (7 downto 0)

BRAM PARA OS BITS 31 a 24

```
-- bits 31 downto 24
```

```
block_3: RAMB16_S9
    generic map (
        INIT 00 => X"0000000000000000000002ab00000000000000000100000000000000000000000"
```

Memórias no FPGA – Dados

```
-- bits 7 downto 0
--
block_0: RAMB16_S9
  generic map (
    INIT_00 => X"1605106231781403f402021605106231781403ff110000000000000000000000",
    INIT_01 => X"000000000000000000000000000000000000000000000000000000000000b20",
    INIT_02 => X"000000000000000000000000000000000000000000000000000000000000",
    INIT_03 => X"000000000000000000000000000000000000000000000000000000000000",
    ...
    INIT_3E => X"000000000000000000000000000000000000000000000000000000000000",
    INIT_3F => X"000000000000000000000000000000000000000000000000000000000000")
  port map (
    CLK      => clock,
    ADDR     => address,
    EN       => '1',
    WE       => we0,
    DI       => d_in_mem0,
    DIP      => "0",
    DO       => data_out(7 downto 0),
    SSR      => '0'
  );
end data_mem_mod0;
```

- Exemplo de configuração de uma BRAM (do exemplo SomaVet.asm) com os bytes menos significativos de cada palavra (bits 7 a 0 ficando no bloco 0 de BRAM, Memória de Dados)

Como Gerar o VHDL destas Memórias?

- Usando o programa **le_mars** (código fonte e executável [Windows] são fornecidos)

- Executando **le_mars**

```
*****
*      PROGRAMA QUE LE DOIS ARQUIVOS: instruções e dados. Uso:      *
*      ./le_mars <arquivo programa> <arquivo dados> [ni] [nd] [d]    *
*      ...                                                            *
*      [d] pode ser qualquer caractere, sendo opcional, indica debug *
*      Report all bugs to ney.calazans@ufsc.br, fernando.moraes@pucrs.br *
*****
```

- Executando: **./le_mars** prog.txt data.txt 1 4

```
----- PROCESSING INSTRUCTIONS
----- PROCESSING DATA
----- GENERATING VHDL FILE    memory.vhd
```

- Arquivo **memory.vhd** gerado

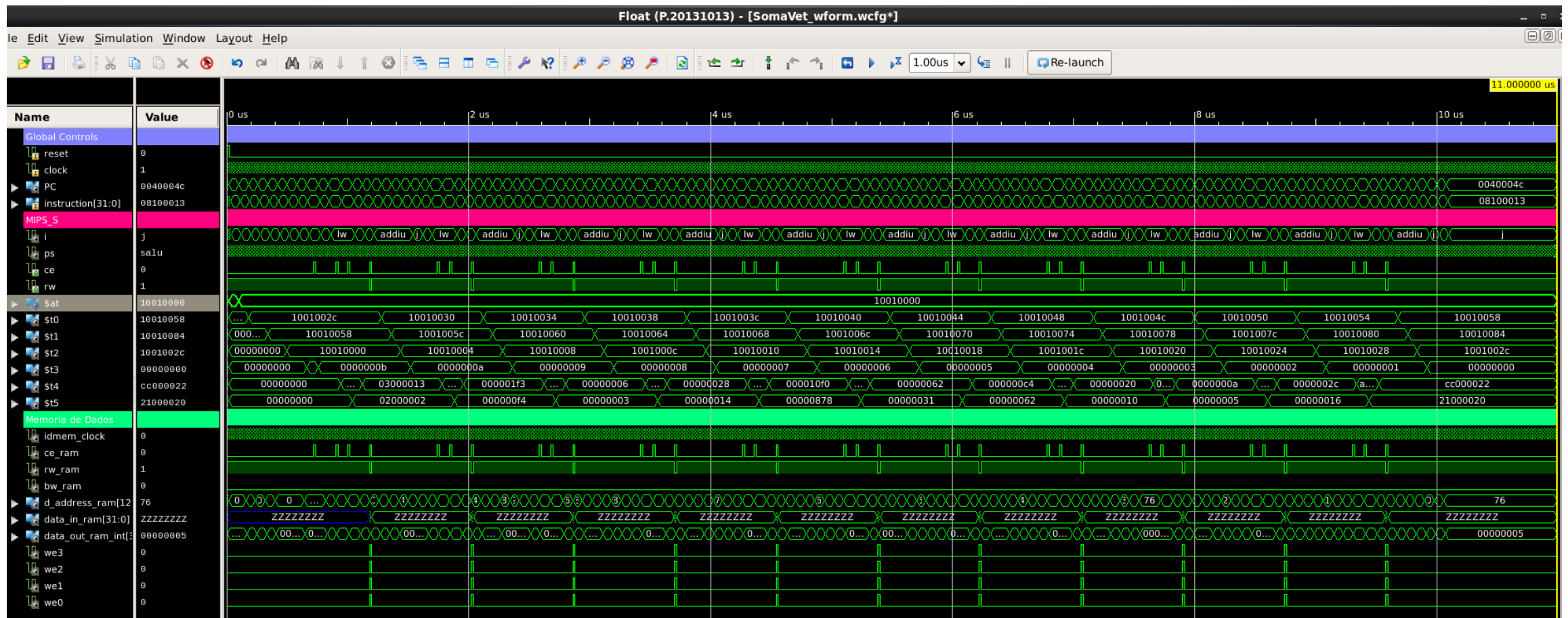
Simulando o Sistema MIPS_S_withBRAMS

- Criar um projeto no ISE com 6 arquivos
 - ✓ `memory.vhd` - gerado pelo programa `le_mars.exe` a partir dos segmentos de texto e dados do MARS contendo o código objeto e dados de entrada do programa `SomaVet.asm`
 - ✓ `MIPS_S_Prt.vhd` - o principal arquivo que descreve o processador MIPS_S
 - ✓ `mult_div.vhd` - arquivo adicional, contém o Hw do multiplicador e do divisor, instanciados no MIPS_S
 - ✓ `MIPS_S_withBRAMS.vhd` - o sistema processador+memórias+lógica de cola
 - ✓ `mips_sem_perif_tb.vhd` - um testbench básico para o sistema MIPS_S_withBRAMS
 - ✓ `SomaVet_wform.wcfg` - um arquivo de organização de sinais a visualizar durante a simulação do programa `SomaVet`
- Simule o sistema por **10.3 μ s** (microssegundos), para um clock de 50 MHz. **Metade deste tempo, se o clock for de 100MHz!**



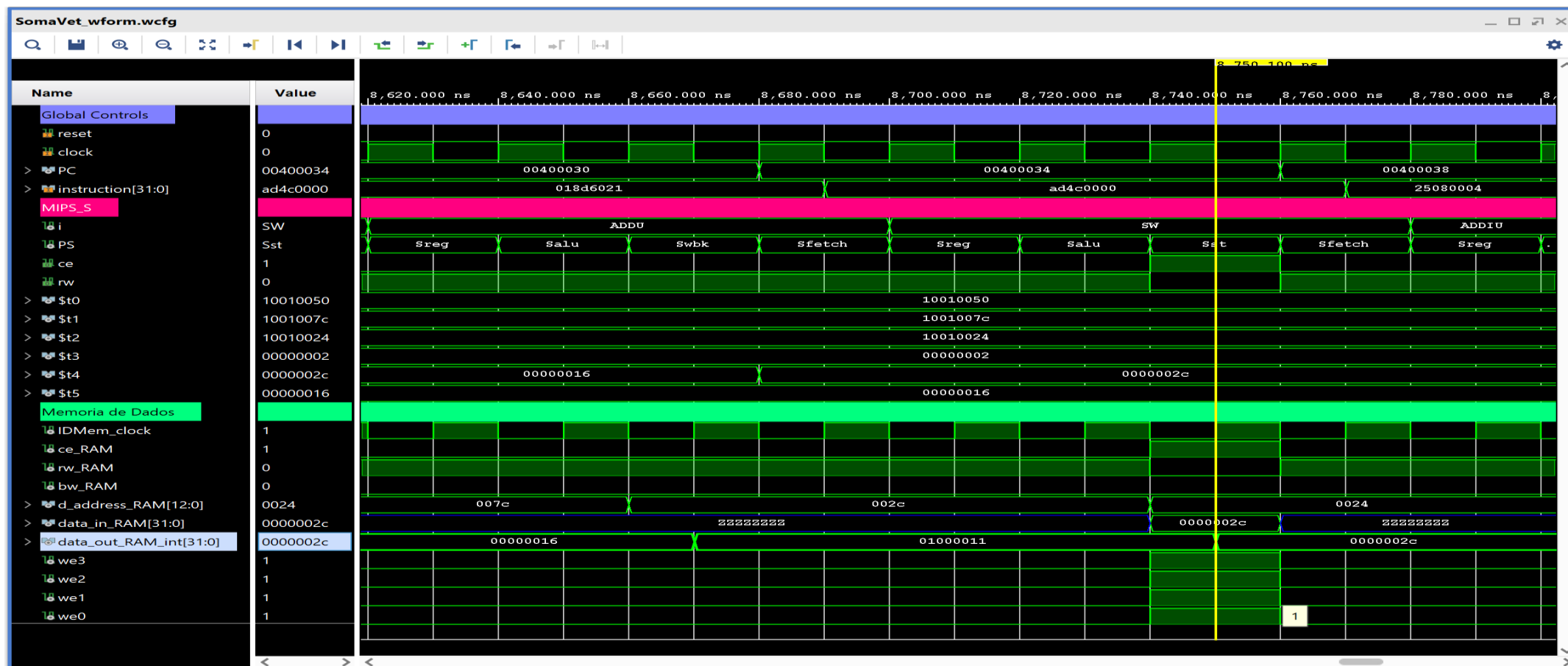
Visão Macro da Simulação

- Pode-se perceber as 22 leituras e as 11 escritas na Memória de Dados, observe-se os sinais **ce**, **rw**, **ce_RAM**, **rw_RAM** e **we3** a **we0**



Detalhe de uma Escrita na Memória

- Instrução de escrita (**sw**), leva 4 ciclos (busca/busca operandos/calc end./escrita mem.)
- Escrita do valor 0x0000002c (reg \$t4) a partir do endereço 0x0024 das memórias de dados



Resumindo o Processo

1. Carregar/simular/validar no MARS um programa escrito em linguagem de montagem
 - ✓ No exemplo, o [SomaVet.asm](#)
2. Fazer os *dumps* de memória (tipicamente, os *dumps* são realizados após a montagem do programa e antes de executar a simulação)
 - ✓ [prog.txt](#)
 - ✓ [data.txt](#)
3. Gerar o [memory.vhd](#) a partir destes dois arquivos, usando o programa *le_mars*
4. Simular projeto contendo os arquivos fonte dados na distribuição, quais sejam
 - ✓ [MIPS_S_sem_perif_tb.vhd](#) → Testbench do Sistema MIPS_S_withBRAMs, sem periférico
 - ✓ [MIPS_S_withBRAMS.vhd](#) → O Sistema MIPS_S_withBRAMs em si
 - ✓ [MIPS_S_Prt.vhd](#) → O Processador MIPS_S prototipável
 - ✓ [multi_div.vhd](#) → Operadores de multiplicação e divisão (usados no MIPS_S_Prt)
 - ✓ [memory.vhd](#) → Memórias de Instruções e Dados com o programa e os dados de execução
 - ✓ [SomaVet_wform.wcfg](#) → Organização de sinais a visualizar durante a simulação

ENUNCIADO DO TRABALHO T4

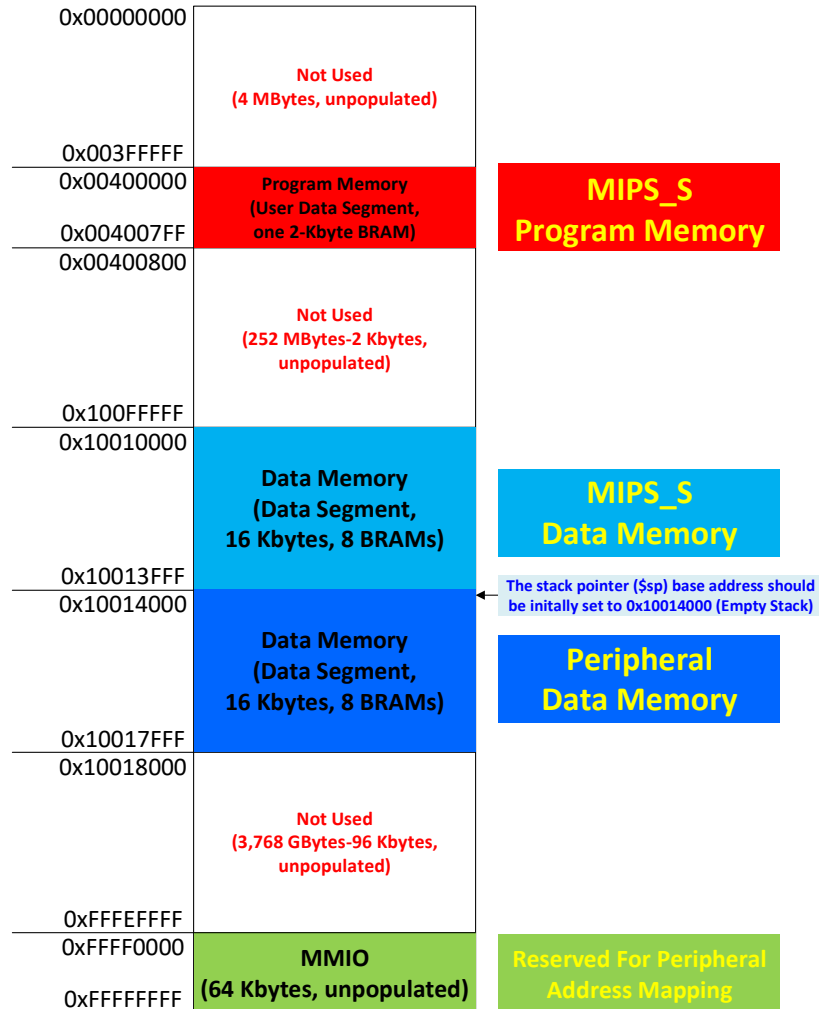
INTRODUÇÃO AO T4

O T4 compõe-se de 4 Projetos, cujo objetivo geral é lidar com um sistema computacional complexo (MIPS_S_withBRAMs) e desenvolver o hardware de um Periférico capaz de interagir com este. A abordagem incremental é

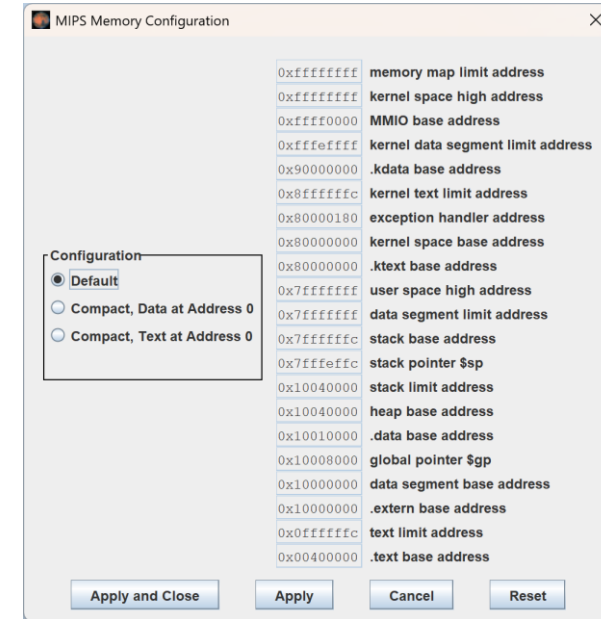
1. Projeto 1 – Repetir os passos de como simular o Hw do sistema MIPS_S_withBRAMs executando uma aplicação (Slides 7 a 19 acima), para dominar a estrutura do processo
2. Projeto 2 – Desenvolver um software capaz de transferir uma quantidade grande de dados de um arquivo para a memória de um periférico e daí copiá-los para a memória de dados do sistema MIPS_S_withBRAMs, gerando a estrutura de memória a usar nos Projetos 3 e 4
3. Projeto 3 – Com a estrutura de memória montada no Projeto 2, simular o Sw (devidamente modificado) sendo executado pelo Hw do sistema, avaliando sua execução
4. Projeto 4 – Criar um Periférico e um módulo Topo. O primeiro deve se plugar na interface do MIPS_S_withBRAMs, conter uma máquina de estados, uma memória local, e interfaces para dispositivos de entrada e saída. O Periférico deve permitir: (1) transferir dados da sua memória local para a do o sistema; (2) Permitir ao usuário externo verificar a transferência

O MAPA DE MEMÓRIA DO MIPS_S A USAR

The MIPS_S Memory Map for the MIPS_S_withBRAMs system. This map is based on the Default Memory Model from the MARS Simulator Environment and is supported by MARS. The size of the memory pieces are not drawn proportionally.



1. O MIPS_S, como o MIPS, usa endereços de 32 bits, cada um destes sendo o endereço de 1 byte
2. O MARS usa como modelo *default* um mapa de memória com diversos segmentos →→→→→→→→
3. O MIPS_S usa apenas 3 destes segmentos: 1 para a Memória de Programa (2KBytes), 1 para a Memória de Dados (32Kbytes divididos entre o MIPS_S e o Periférico) e 1 segmento para mapeamento de dados do Periférico
4. À esquerda se mostra a estrutura do mapa do MIPS_S - regiões de memória realmente utilizáveis são coloridas
5. *unpopulated* → não existe memória real na região



PROJETO 1 (1)

1. Repitam todo o processo descrito nos slides anteriores para executar o programa exemplo *SomaVet.asm* no MIPS_S prototipável, usando simulação de hardware (e.g. com o ISIM)
 - Partam do programa em linguagem de montagem (*assembly language*, **.asm**), estudem-no e entendam ele
 - Simulem o mesmo, usando o simulador do MIPS, o MARS
 - Gerem os *dumps* dos segmentos de texto e de dados, conforme mostrado
 - Rodem o programa **le_mars** sobre os *dumps*, para gerar o arquivo **memory.vhd**

PROJETO 1 (2)

- **Atenção:** Nesta versão do sistema **MIPS_S_withBRAMs** usa-se uma Memória de Dados de 16Kbytes, composta por 2 módulos de memória entrelaçada com 4 BRAMs cada. O segundo módulo não é usado pelo *SomaVet.asm*, mas será útil mais tarde. A geração com *le_mars* usa os parâmetros opcionais *ni=1* e *nd=2* ou 4
- Gerem um projeto no ISE usando os arquivos VHDL do ambiente (como mostrado) e com o **memory.vhd** gerado
- Dominem neste projeto o processo de usar o ambiente de simulação do processador MIPS_S com blocos de memórias (BRAMs), o **MIPS_S_withBRAMs**

PROJETO 1 (3)

2. No material de apoio, há um documento de referência que ajuda para realizar este projeto, com mais detalhes que aqui
 - ✓ [5_5-Simulacao_MIPS_S_withBRAMs_V5.9.pdf](#)

PROJETO 2(1)

1. Escrevam um Sw em linguagem de montagem do MIPS capaz de transferir texto de uma região de memória para outra de mesmas dimensões, similar à função **strcpy** da linguagem C. Para tanto, assumam que
 - a. Tem-se disponível um ponteiro (ou seja, um endereço de memória) para a **área de memória fonte** dos dados a copiar e um ponteiro para o início da **área de memória destino** dos dados. Os dados terminam com um caractere NULL (0x00)
 - b. A **área de memória destino** tem espaço suficiente para receber os dados copiados
 - c. A **área de memória destino** está na Memória de Dados do MIPS_S (que possui 16.384 bytes [16KB]), ocupando os endereços de 0x10010000 até 0x10013FFF, inclusive
 - d. A **área de memória fonte** está em outro lugar da Memória de Dados do MIPS_S (tipicamente em módulo(s) disponível(is) no arquivo **memory.vhd**, mapeada para endereços que o MARS pode usar). Estes devem ser compatíveis com uma região de memória a ser usada pelo Periférico, que futuramente estará conectado ao sistema MIPS_S_withBRAMs (no Projeto 4). **Use a região do mapa do MIPS a ocupando os endereços de 0x10014000 a 0x10017FFF**
 - e. O Periférico mapeia seus registradores locais para acesso pelo Sw do MIPS_S **usando endereços da região MMIO, entre os endereços 0xFFFF0000 a 0xFFFFFFFF**

PROJETO 2(2)

Observação: Este arquivo tem de estar no mesmo diretório do executável do MARS

2. Validem o programa do item 1. do Projeto 2 usando o simulador MARS
- Usem o arquivo fornecido (**Romeo&Juliet_Act2-Scene2.txt**) como entrada textual, empregando chamadas do sistema do Mars (*system calls*, em particular as chamadas de números 13, 14 e 16, respectivamente para abrir, ler e fechar arquivos)
 - Usem a área de dados abaixo para o programa a desenvolver

```
.data          # The MARS data space starts at address 0x10010000
dest_text_addr: .word      0
                .space     9200  # This completes the reserved quasi-9Kbytes
                                # space for the destination string
after_dest:    .word      0      # After the destination string end, put a label,
                                # to enable visualizing the end of the MIPS_S Data Memory
                .space     7144  # Jump to get close to address of the 16Kbytes block of the Peripheral Memory
filename:      .asciiz     "Romeo&Juliet_Act2-Scene2.txt"  # filename at the end, to facilitate..
source_text_addr: .word     0    # After the filename end, put label, which must start at address 0x10014000
                                # to enable visualizing the end of the data segment window
                .space     9200  # This completes the reserved quasi-9Kbytes for the source string
after_source:  .word      0      # After the source string end, put a label,
                                # to enable visualizing the end of the data segment window
```

- Desenvolvam um programa para realizar a função *strcpy*, mas que antes chame uma subrotina (usando JAL) que carrega o texto na memória, a partir do ponteiro `source_text_addr` (que aponta para 0x10014000 no MARS)

PROJETO 2 - APÊNDICE

TEXTOS GRANDES OU GRANDES TEXTOS?

- A ideia deste trabalho → lidar com textos grandes, mas também com **Grandes Textos** 😊
- Em particular usamos um estudo de caso - Cena de um Ato de peça de W. Shakespeare
 - William Shakespeare (1564-1616) escreveu, entre 1589 e 1612 (ver o site [Open Source Shakespeare](#) da University George Mason)
 - 37 peças (14 comédias, 12 tragédias e 11 peças históricas), 154 sonetos e 5 poemas (com entre ~100 a ~2.000 versos cada)
 - A peça aqui é [The Tragedy of Romeo and Juliet](#) (5 Atos); trabalha-se a [Cena 2 do Ato II](#)
- Shakespeare segue a tradição clássica da Grécia Antiga, veja-se a definição de [Musas](#)
- Porque Shakespeare? Pelo enorme impacto no mundo da literatura (em particular no Ocidente). O teatro aparece pouco no Brasil, comparado com outras formas de texto
- Outras possibilidades para explorar peças e poemas seriam o mundo do alemão Bertholt Brecht ([Die Dreigroschenoper](#), [Leben des Galilei](#), [Mutter Courage und ihre Kinder](#)), autores franceses como Molière ([L'École des femmes](#), [Le Bourgeois gentilhomme](#)) e [teatrólogos nossos, brasileiros](#), como o clássico Martins Pena e mais recentes, como Ariano Suassuna, Nelson Rodrigues e Chico Buarque

PROJETO 3 (1)

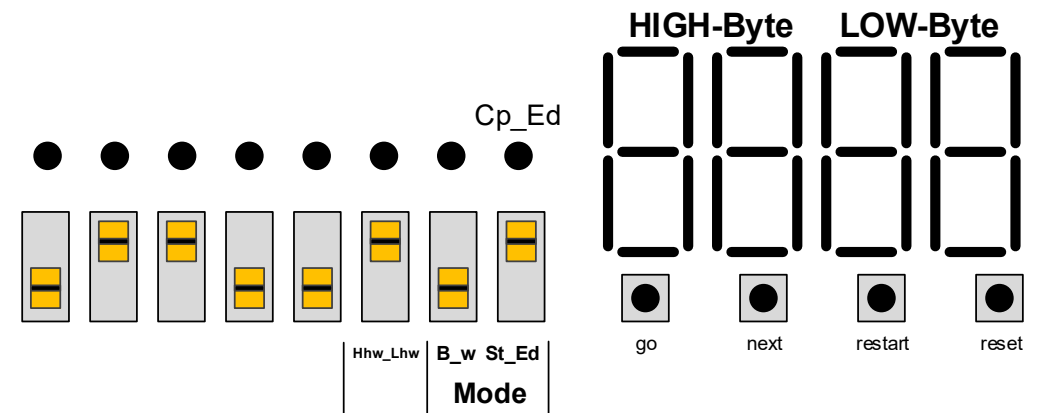
1. Depois de validar o programa no MARS, alterem ele e montem um ambiente de simulação do sistema MIPS_S_Sim, a exemplo do que foi feito para o programa exemplo **SomaVet.asm** com o sistema MIPS_S_withBRAMs. As alterações no programa são
 - a. Não usem a rotina de leitura do arquivo para a memória (a memória será pré-carregada de outra forma, pois o MIPS_S não consegue, ainda, processar arquivos...)
 - b. Alterem o final do programa, para que este termine com um loop eterno, do tipo
aqui: j aqui
Isto facilita a visualização do fim da simulação...
2. Desta vez, não usem os arquivos do ambiente MIPS_S_withBRAMS, mas o ambiente com a versão simulável do processador, conteúdos do diretório **2_MIPS_S_Sim**
3. Leiam o arquivo **README.txt** deste diretório, para entender como configurar o *testbench* VHDL dado e simular seu programa no MIPS_S
4. Vejam no slide a seguir alguns cuidados de como preparar a simulação
5. Tendo dificuldades para fazer a simulação, consultem o professor...

PROJETO 3 (2)

1. O programa MARS na versão mais recente (V. 4.5) possui algumas características indesejáveis (limitações e/ou *bugs*) para realizar o *dump* de áreas de memória. Isto é particularmente sério no caso de áreas de memória grandes
2. Como no Projeto 2 usa-se 32Kbytes de memória de dados, estas características/*bugs* atrapalham um pouco. O *dump* da área de texto (o programa) pode ser feito como mencionado antes, mas o *dump* da área de dados necessita de tratamento diferenciado
 - a. Primeiro, carregue no MARS o programa gerado para este projeto como descrito no slide intitulado [Projeto 3\(1\)](#). Faça o *dump* da área de texto como explicado antes
 - b. Segundo, carregue no MARS o programa gerado no [Projeto 2](#), execute-o e só então faça o *dump* da área de dados como explicado antes. Isto deve gerar um *dump* da área de dados completa (de 0x10010000 a 0x10016FFF)
 - c. Terceiro, edite o arquivo data.txt (ou o nome usado para o *dump*), eliminando os dados da cadeia nas primeiras 287 linhas deste (que contêm o texto já copiado), trocando cada palavra nesta região para 0x00000000. Ajuda nisto usar um editor com capacidade de seleção de qualquer conjunto de linhas e colunas de texto, como o Notepad++. Eu copiei os dados (sem incluir os endereços no início de cada linha), das linhas 301-500 e sobrepos estes dados nas linhas 1-200. Depois copiei o quadro das linhas 301-387 e sobrepos estes dados nos dados das linhas 201-287. Outras soluções existem, claro. Salve o novo data.txt e continue a partir do item 3 do slide anterior (intitulado [Projeto3\(1\)](#))...

PROJETO 4 – Especificação (1)

1. Acrescentar um periférico (hardware) ao sistema MIPS_S_withBRAMs. Este conterá lógica para realizar a transferência do texto longo do arquivo (**Romeo&Juliet_Act2-Scene2.txt**) sob comando de um usuário externo. O texto deve estar devidamente carregado na memória física do periférico. O periférico copia este texto para a memória de dados interna do sistema MIPS_S_withBRAMs. Como realizar isto?
 - a) A interface do Periférico com o mundo externo → deve ser aquela dada abaixo, que usa os 4 *push-buttons*, 3 das chaves deslizantes, 1 *led* e o *display* da placa Nexys 2, com as funções
 - reset – botão de inicialização geral do sistema (módulo Topo)
 - go, next, restart – controle de visualização do texto copiado (**detalhes adiante**)
 - Cp_Ed – *Led* que avisa quando a cópia do texto pelo Periférico foi concluída
 - Mode – duas chaves que controlam o modo de visualização – B_w escolhe ver byte a byte (SW1 em 0) ou palavra a palavra (SW1 em 1); St_Ed escolhe se visualização parte do início do texto (SW0 em 0) ou do fim do texto (SW0 em 1)
 - Hhw_Lhw – chave que controla o que se vê no display no modo palavra a palavra – SW2=0 indica que o display mostra a meia palavra baixa, SW2=1 indica que a meia palavra alta é visualizada



PROJETO 4 – Especificação (2)

- b) É fornecido o esqueleto do Periférico a implementar (uma entidade e uma arquitetura parcial), contendo uma interface com o sistema MIPS_S_withBRAMS, uma memória preenchida com o texto e localizada em endereços mapeados a partir da posição 0x10014000 no mapa de memória do MIPS_S e uma interface com dispositivos da placa Nexys 2 mencionados (*push-buttons, led, chaves, display*)
- c) Com o esqueleto dado em b), deve-se desenvolver uma máquina de estados que
 - i. Ao ser ressetada, aguarda ordem do usuário (botão 'go' apertado) para realizar a cópia do texto de sua memória para a do processador
 - ii. Avisa MIPS_S que 'go' foi apertado (posição de memória legível) e suspende a operação do MIPS_S para a transferência
 - iii. “Des-avisa” que 'go' foi apertado e realiza transferência, armazenando o tamanho em “text_size”
 - iv. Ao final da cópia avisa fim ao MIPS_S (via posição de memória legível por ele) e ao usuário (Led CP_Ed acende); em seguida aguarda 'go' apertado e lê 'Mode' das chaves, setando seu valor e ligando 'go' novamente para avisar ao MIPS_S
 - v. Des-suspende o MIPS_S. A partir daí, aguarda sequência de comandos de visualização (MIPS_S enviada dados para o display e aguarda comandos 'next' ou 'restart' do usuário)

PROJETO 4 – Especificação (3)

- d) Em paralelo com a máquina de estados do periférico, o MIPS_S executa um Sw (a ser desenvolvido neste trabalho). A estrutura geral deste compõe-se de quatro laços em que o processador interage com o periférico
- i. Após ser inicializado o MIPS_S entra no **laço 1**, lendo a informação sobre o botão 'go' ter sido apertado ou não. Sai-se do laço ao detectar o apertar do botão
 - ii. Após sair do **laço 1**, o MIPS_S entra no **laço 2**, onde aguarda o fim da cópia do texto para a sua memória local pelo Periférico, que deve suspender o MIPS_S para isto
 - iii. Ao sair do **laço 2**, isto significa que a cópia foi realizada, e que é o modo de visualização já foi definido e o botão 'go' foi novamente apertado. O MIPS_S então executa o **laço 3**, onde lê o fato de 'go' ter sido apertado junto com o valor de 'Mode'. Quando isto acontece, MIPS_S sai deste laço.
 - iv. Com o valor de 'Mode', o MIPS_S define o endereço de início do texto copiado, talvez lendo o "text_size" do Periférico (se St_Ed='1'), lê primeiro dado (byte ou word) e envia para o Periférico, setando 'Show' para '1'
 - v. Em seguida, o MIPS_S entra no **laço 4**, onde aguarda o apertado de um dos botões 'next' ou 'restart' para re-enviar novo dado (a partir do início se 'restart' foi apertado, senão incrementado o ponteiro de leitura (de 1 em 1 ou de 4 em 4, dependendo se B_w está em '0' ou '1', respectivamente)

PROJETO 4 – Dicas (1)

1. Usem a especificação para definir a estrutura da do Periférico e do Sw
2. A comunicação entre Sw (MIPS_S) e Hw (Periférico) deve usar técnicas de **entrada e saída (E/S) mapeada em endereços** (para o MIPS_S acessar dados do periférico) e acesso à memória do MIPS_S pelo periférico via **suspensão do MIPS_S** (para realizar a cópia do texto do Periférico para a memória do MIPS_S). Ver sinais do MIPS_S ‘suspend’ e ‘suspend_ack’
3. Da especificação do sistema, os endereços que o Sw deve usar para buscar ou enviar dados ao Periférico são 5 para leitura e 2 para escrita
 - i. Leitura
 - o valor do botão ‘go’ (1 bit)
 - o fato da cópia haver terminado ‘Cp_Ed’ (1 bit)
 - o modo de visualização ‘Mode’ (2 bits),
 - o fato de um dos botões next ou restart ter sido apertado ‘nxt_rst’ (2 bits)
 - o tamanho do dado copiado “Text_size” (14 bits, no mínimo)
 - ii. Escrita
 - valor a ser mostrado no display, digamos ‘Current_chunk’ (com 32 ou 8 bits relevantes)
 - ordem do MIPS_S para habilitar que os dados sejam mostrados no display, ‘Show’ (1 bit)

PROJETO 4 – Dicas (2)

4. Cada um dos endereços corresponde a um registrador no Periférico (7)
5. Sugere-se usar aqui endereços da região MMIO (facilita depurar o Sw no Mars), 1 para cada valor, e.g. 0xFFFF0000 para 'Go', 0xFFFF0001 para 'CP_Ed' etc. É bom definir os endereços na área de dados do Sw, de forma simbólica
6. Quando 'Mode' determina leitura palavra a palavra, lê-se 32 bits de texto por vez (4 caracteres ASCII), mas o display só pode mostrar 2 bytes de cada vez (2 caracteres ASCII em hexa). Assim, usa-se um multiplexador 2x(16 bits):1x(16bits) na saída, a chave '**Hhw/Lhw**', dizendo o que deve ir no display
7. Quando se usa leitura palavra a palavra do texto a partir do fim do texto ('Mode'=11), o Sw precisa tratar um possível caso especial na primeira leitura, pois a instrução **lw** do MIPS só pode ler de endereços múltiplos de 4.

Cuidado!

PROJETO 4 – Dicas (3)

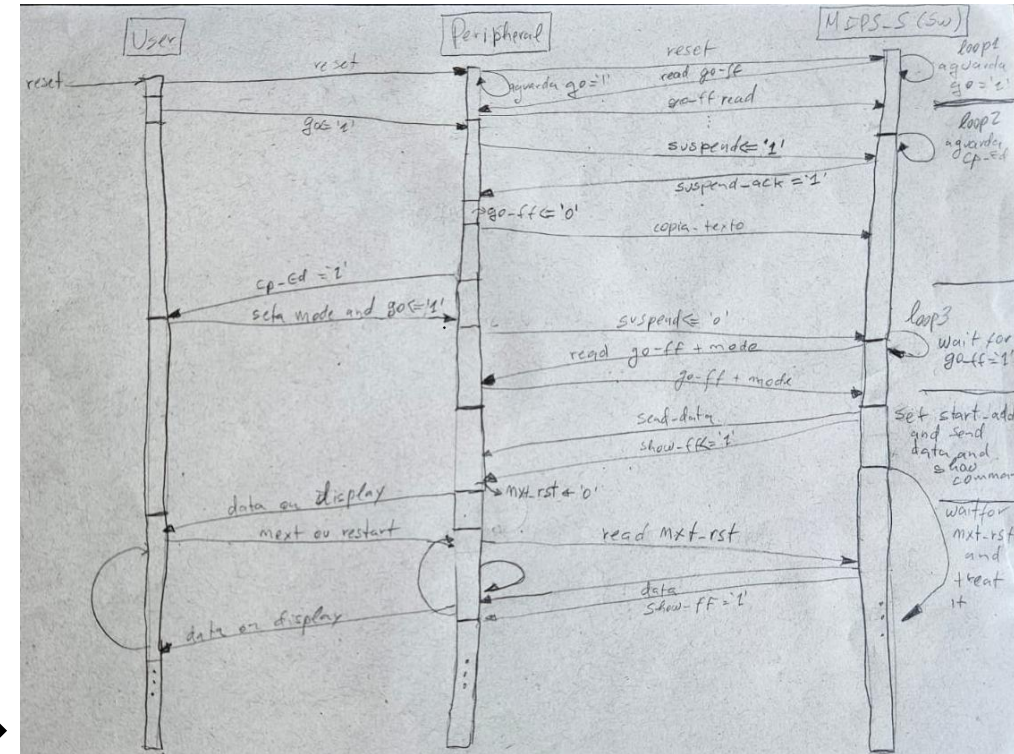
8. Para uniformizar a maneira de escrever o Sw a rodar no MIPS_S e facilitar a sua depuração usando o Mars, usem a área de dados dada abaixo

```
# Data segment
                                .data
MIPS_S_text_start: .word      0
                                .space 9200                # This completes the reserved quasi-9Kbytes
                                                                # space for the destination string
after_dest:         .word      0
                                                                # After the string end, put a label,
                                                                # to enable visualizing the end of the data segment window
                                .space 7144                # jump to align close to address 0x10014000

# MMIO Addresses
go_ff_addr:         .word      0xFFFF0000                # Address where to read the go ff store (bit 0)
cp_finished_ff_addr: .word      0xFFFF0001                # Address where to read the copy finished ff store (bit 0)
mode_reg:           .word      0xFFFF0002                # Address where to read the copy finished ff store (bits 1-0)
nxt_rst_reg:        .word      0xFFFF0003                # Address of peripheral register containing info about
                                                                # mode switches byte-word (B_w) and start-end (s_e
text_size_reg:      .word      0xFFFF0004                # Address where to read the text size (14 bits 13-0), multiple of 4
data_ts_reg_addr:   .word      0xFFFF0008                # Address where to write value of the text data (32 bits)
                                                                # This is address must be a multiple of 4
show_ff_addr:       .word      0xFFFF000C                # Address where to write value of the show ff store (bit 0)
adjust_line:        .word      0                        # This is an unused data to align the start of the text
# Below is the memory space reserved to store the source text, starting at address 0x10014000
source_text_addr:   .word      0                        # Here starts the 2nd half the memory (second 16Kbytes chunk)
                                .space 9200                # This completes the reserved quasi-9Kbytes for the source string
after_source:       .word      0                        # After the string end, put a label,
                                                                # to enable viewing the end of the text in the Peripheral Memory
```

PROJETO 4 – Dicas (4)

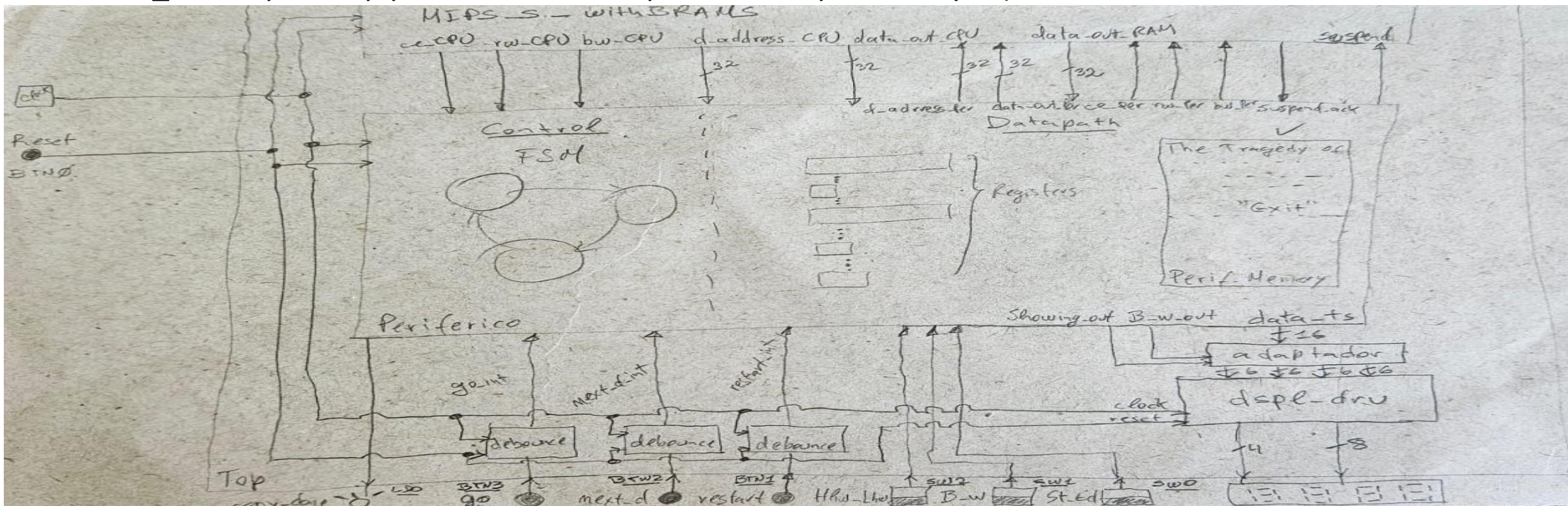
9. A estrutura dos dados nas Memórias de Dados do MIPS_S e do Periférico deve ser aquela obtida no Projeto 3 (ver item 2. do slide 30 [Projeto 3 \(2\)](#)) para obter o arquivo data.txt (*dump* do **Data Segment**). Com o Sw do MIPS_S desenvolvido, o prog.txt deve ser apenas o *dump* do do **Text Segment** deste
10. Para ajudar a elaboração do sistema, segue ao lado um esboço grosseiro de um Diagrama de Sequência para o sistema, mostrando as ações do Usuário, do Periférico e do MIPS_S (Sw) →→→→→→→→→→



Bad News → A quantidade de BRAMs exigida neste projeto (17) não permite que o FPGA da Nexys 1 (XC3S200) seja usado, pois ele só possui 12 BRAMs – Para prototipar o sistema, compartilhem as Nexys 2 com grupos que têm ela...

PROJETO 4 – Dicas (5)

11. O esqueleto do código VHDL do Periférico fornecido no Moodle tem a estrutura descrita graficamente com o desenho abaixo, com 2 interfaces (com o módulo Top e com o ambiente MIPS_S_withBRAMs) e 3 estruturas principais internas ao Periférico (1 **Memória de Dados**, dada de forma completa no esqueleto, 1 **FSM** a ser desenvolvida e 1 **conjunto de registradores** que armazenam informação utilizável pelo Sw do MIPS_S e/ou pelo Top para acionar os dispositivos da placa Nexys 2)



RESUMO DO TRABALHO T4

- O **Trabalho T4** a ser entregue consiste em um arquivo compactado (.zip) contendo
 - ✓ Um relatório em PDF descrevendo os Projetos 2-4 (2 pontos)
 - ✓ Códigos fonte *assembly* gerados, comentados (Projeto 2) (2 pontos)
 - ✓ Diretório com os VHDLs (Projetos 3 e 4), arquivos .ucf e .bit (Projeto 4) (3 pontos)
 - ✓ O projeto completo ISE/Vivado do protótipo gerado (Projeto 4) (3 pontos)
 - ✓ **DEVE-SE** mostrar a operação do projeto na plataforma de prototipação ao professor em alguma ocasião, ou enviar um vídeo onde a operação correta é demonstrada